

A Developers' Guide for the *bsc* compiler

Authors: Various (see end of §1)

Version: June 11, 2026

Abstract

This document is intended to assist people who are studying or modifying the Haskell source code of the *bsc* compiler, including developers (maintainers/bug-fixers/improvers) and PR-reviewers.

It may also be useful for others who wish to instrument *bsc* for analysis and/or connection to other tools (*e.g.*, formal verification tools, LSP servers, code browsers, ...).

We assume the reader has a basic knowledge of Haskell (nothing advanced). §A is a brief cheat sheet.

Contents

1	Overview	2
2	Resources	2
3	Notation in this document	3
4	Top-level of the <i>bsc</i> compiler	3
4.1	<code>Main_bsc::main</code> , <code>Main_bsc::hmain</code> and <code>Main_bsc::hmain'</code>	4
4.2	Compiling a package (file) <code>Main_bsc::compilePackage</code>	4
4.3	Creating (and re-creating) the symbol table <code>MakeSymTab::mkSymTab</code>	6
4.4	Handling infix/prefix ops and associativities <code>ParseOp::parseOps</code> . . .	6
4.5	Annotating module input/output ports <code>GenWrap::genWrap</code>	6
5	Major internal representations of the program	7
5.1	Concrete Syntax <code>CSyntax</code>	7
5.2	Intermediate Syntax <code>ISyntax</code>	7
5.3	Abstract Transition Syntax <code>ASyntax</code>	7
6	Parsing <code>Depend::parseFile</code>	7
6.1	Parsing BH <code>CParser::pPackage</code>	8
6.2	Parsing BSV <code>CVParser::bsvParseString</code>	8
7	Type checking <code>TypeCheck::cTypeCheck</code>	8
8	Code generation <code>Main_bsc::genModule</code>	9
9	Notes on building the <i>bsc</i> compiler (compiling and linking)	10
A	Brief Haskell cheat sheet	11

1 Overview

The *bsc* compiler is invoked like any Unix executable from the shell command-line with command-line flags and arguments. The `bsc` command actually invokes a small script that ultimately invokes the main compiler binary. “`bsc -help`” will list the command-line flags and arguments.

The *bsc* compiler is written in Haskell, and is compiled to a binary using the **ghc** Glasgow Haskell Compiler.

The entry point of any Haskell program is the function `main` (§4.1).

The main compiler flow is in the `compilePackage` Haskell function. It is structured as a sequence of *stages* (or compiler passes), described in more detail later in §4.2 and illustrated in Figure 1.

Each stage transforms a data structure input to a data structure output, and these are passed sequentially from stage to stage. The data structures are an internal representation of the **BSV**/**BH** program, plus auxiliary information.

As indicated in the figure, the intermediate data structures are of three major types:

CSyntax	Concrete Syntax, augmented with type information (§5.1)
ISyntax	Intermediate representation (§5.2)
ASyntax	Abstract Transition Syntax (§5.3)

Authors of this document This is an open-source document and is expected to be updated by various people to keep it in sync with the *bsc* compiler as *bsc* evolves over time. The git log should list the contributors and their contributions to this document. This document was begun by Rishiyur S. Nikhil on 2026-06-10, as part of his own journey to learn the internals of *bsc* (to that date he was not an author any of the *bsc* compiler source code).

2 Resources

The *bsc* compiler is in this repository (repo):

<https://github.com/B-Lang-org/bsc> The compiler is written in fairly vanilla Haskell and exists in a collection of Haskell source code files (`.hs` and `.lhs` extensions). The compiler, per se, is mostly in the `src/comp/` directory; the **BSV** and **BH** parsers are in `src/comp/Parser/`.

In the repo, the file `INSTALL.md` describes how to build (compile and link) the *bsc* compiler. *bsc* is built using the **ghc** Haskell compiler. This document also has some notes on building, in §9.

NOTE:

Most *users* of *bsc* do not need to build *bsc* from scratch. Pre-built binaries are available for many Linux platforms and for MacOS; please see the “Download ... Releases” link in the repo’s README.

3 Notation in this document

Each listing display in the sections that follow is a gloss of a Haskell function in the source code, *i.e.*, a summary, omitting much detail. These are intended as a high-level “road map” for the actual code, are best read side-by-side with the actual code.

Our focus in these listings is on the “flow” of major functions, and so we omit the actual “plumbing” that connects inputs and outputs of functions. Haskell being a pure functional language, all plumbing is with explicit inputs and outputs (no side-effects on external data structures). Function inputs are arguments “`f x`”, and functions outputs are typically bound with “`let y = f x`” let-bindings, or “`y <- f x`” monadic bindings, or directly returned into a surrounding expression. The details of plumbing values between functions are best seen in the actual source code.

Many listings begin with the name of the file containing the code.

File: `bsc/src/comp/bsc.hs`

Haskell identifiers are shown as is, “`x`”, if the Haskell file/module context is obvious, or prefixed with its Haskell module name to clarify, “`module_name:x`”.

NOTE: A possible point of confusion: What Haskell calls a **module** (a large-program organizing construct) is called a **package** in **BSV/BH** because in **BSV/BH** the word “module” is reserved for hardware items.

Function call chains are shown indented like this (`f1` calls `f2` which calls `f3` which calls ...):

```
f1
  f2
    f3
      ...
```

Functions called in sequence (in a monadic `do`) are listed vertically in sequence, like this:

```
f1
f2
...
```

Functions called as alternatives (in if-then-elses/cases) are listed vertically in sequence with vertical bars, like this:

```
| f1
| f2
| ...
```

A list of items is shown using the usual Haskell square-bracket list notation: `[item]`

4 Top-level of the *bsc* compiler

The entry point for the *bsc* compiler program, like any Haskell program, is the function `main` (§4.1).

4.1 Main_bsc::main, Main_bsc::hmain and Main_bsc::hmain'

`main` just invokes `hmain` within a wrapper that catches any exceptions raised therein.

`hmain` performs one of several top-level actions depending on command-line flags; one of those alternatives is to invoke `main'` for compiling a **BSV/BH** program.

File: `bsc/src/comp/bsc.hs`

```
main          — Haskell top-level function
  hmain       — Top-level of bsc, via bsCatch
  | help      — print flags help and exit
  | helpHidden — print flags, hidden flags, and exit
  | usage     — print usage and exit
  | main'     — process a source file
  | vLink ()  — verilog link
  | simLink () — bluesim link
```

`hmain'`, when invoked with `-u`, follows the import chain and locates all files/packages to be recompiled and calls `compilePackage` on each such file.

File: `bsc/src/comp/bsc.hs`

```
main'
  compile_with_deps — when invoked with -u
    chkDeps         — uses Depend::parseFile
                   — and returns [pkg] needing recompilation
    compilePackage  — on each file in [pkg]

  | compile_no_deps — when invoked without -u
    Depend::parseFile — which returns pkg
    compilePackage   — on the specified file
```

See §6 for: `Depend::parseFile`.

4.2 Compiling a package (file) Main_bsc::compilePackage

`compilePackage` embodies the top-level “flow” of the compiler and is a sequential composition of many stages, or passes. These are shown in Figure 1.

Towards the end, a stage invokes `genModule` to actually produce Verilog or Bluesim output. Internally, it converts `ISyntax` into `ASyntax`. `genModule` is discussed in more detail in §8.

Each of the stages is surrounded by `start`, `stats` and `dump` calls. They are meant for providing info to the user on statistics and current state of internal representation. We omit those calls in the gloss below.

File: `bsc/src/comp/bsc.hs`

```
compilePackage
— The incoming BSV/BH package is the final arg
— We are working with CSyntax (incoming package)
  readImports — read signatures of imported packages
  parseOps    — in pkg ParseOp. Handles infix ops
               — (§4.4)
  mkSymTab    — Generate a global symbol table
```

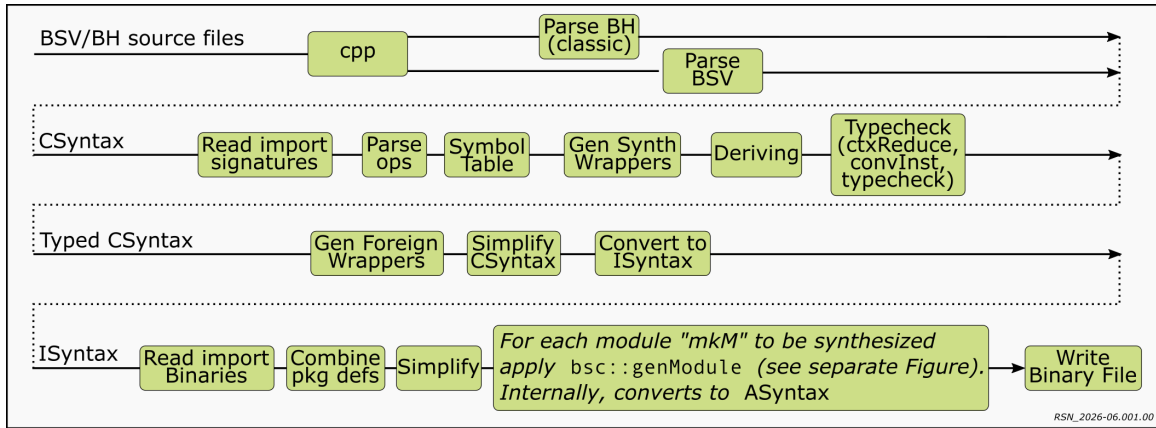


Figure 1: Major stages in flow of *bsc* compiler (see function `compilePacakage`)

```

— (MakeSymTab:: ¶4.3)
— First of several times that the table is built
genFuncWrap — Turn 'noinline' into module definitions
genWrap — Generate wrapper for Verilog interface.
mkSymTab — because added new types (MakeSymTab:: ¶4.3)
— and typeclass instances for those types
addFuncWrap — Re-add function definitions for 'noinline'
derive — Turn deriving into instance declarations
mkSymTab — because Deriving added new instances
— (MakeSymTab:: ¶4.3)
cCtxReduceIO — Reduce the contexts as far as possible
mkSymTab — because possibly changed types of top-level
definitions — (MakeSymTab:: ¶4.3)
cConvInst — Turn instance declarations into ordinary definitions
cTypeCheck — Type check and insert dictionaries
— (TypeCheck:: ¶7)
— We are now working with: Typed CSyntax

genForeign — Generate wrapper info for foreign function imports
— (always happens, even when not generating for
modules)
mkGenFileHeader,
genDPIWrappers — Generate VPI wrappers for foreign function imports
simplify — Simplify a little

iConvPackage — Convert to internal abstract syntax
iPCheck — TODO: What does this do?
— We are now working with ISyntax

map Data.Map.lookup — Read binary interface files
— (just looks up in data structure, not read files?)
map adjEnv — adjust the "raw" packages and then add back their
— signatures so they can be put into the current
— IPackage for linking info.
— Injects the "magic" variables genC and genVerilog
mkSymTab — For "genModule" we construct a symbol table that
— includes all defs, not just user visible defs
— (MakeSymTab:: ¶4.3)

```

```

fixupDefs      — TODO: What does this do?
iSimplify      — TODO: What does this do?
iPCheck        — TODO: What does this do?

orderGens      — TODO: What does this do?

gen            — Generate code for all requested modules (recursive)
  genModule    — Synthesize a module
  compileCDefToIDef
                — TODO: What does this do?
genUserSign    — Generate the user-visible type signature
genEverythingSign
                — Generate a type signature where everything is visible
genBinFile     — Generate binary version of the internal tree .bo file

```

4.3 Creating (and re-creating) the symbol table `MakeSymTab :: mkSymTab`

`mkSymTab` is invoked in `compilePackage` (§4.2). It is invoked multiple times, presumably to keep it in sync with latest transformations to the program representation.

File: `bsc/src/comp/MakeSymTab.hs`

```
mkSymTab :: ErrorHandle -> CPackage -> IO SymTab
```

The symbol table is actually 6 tables: constructor names, type names, ...

4.4 Handling infix/prefix ops and associativities `ParseOp :: parseOps`

`parseOps` handles the various infix, prefix and postfix operators with varying precedences and associativities.

File: `bsc/src/comp/ParseOp.hs`

```
parseOps :: ErrorHandle -> CPackage -> IO CPackage
```

4.5 Annotating module input/output ports `GenWrap :: genWrap`

File: `bsc/src/comp/GenWrap.hs`

```

— creates wrappers for separately compiled modules: wraps pack/unpack
  around method
— arguments and results; surfaces method ready signals
— Pragma are module specific
genWrap :: ErrorHandle -> Flags -> [String] -> Bool -> CPackage -> SymTab ->
          IO (CPackage, [WrapInfo])

```

`genWrap` looks at all modules being synthesized, and for each interface, creates primitive interfaces (bit vectors, prim actions, ...).

It's done before type-checking, but with symbol table info. Question: why is this wrapper generated so early?

This is only the first part. The full wrapper happens after compilation, including scheduling etc.

5 Major internal representations of the program

5.1 Concrete Syntax CSyntax

CSyntax presumably stands for “Concrete Syntax”. This is the “parse tree” representation produced by the **BSV**/**BH** parsers, and is used for the first several stages of the compiler.

5.2 Intermediate Syntax ISyntax

ISyntax presumably stands for “Intermediate Syntax”. and is used in the middle stages of the compiler. This is the representation that is put out into “.bo” intermediate files.

ISyntax is much smaller than CSyntax. It is a “SystemF-ish” representation (*i.e.*, includes type application).

There is a pre- and post-elaboration distinction of ISyntax

Pre: packages Post: modules

5.3 Abstract Transition Syntax ASyntax

ASyntax presumably stands for “Abstract Transition Syntax” (from James Hoe’s thesis), and is used in the final stages of the compiler dealing with rules, scheduling, and for producing the Bluesim and Verilog outputs.

6 Parsing Depend::parseFile

parseFile is invoked in compilePackage (§4.2).

File: bsc/src/comp/Depend.hs

```
parseFile      — run CPP, dump CPP output, parse, check name,
                — dump CSyntax, stats
                — returns a CPackage
  parseSrc      — wrapper to detect file encoding errors
  parseSrc'     — Parse a BH or BSV file
```

The **BSV**/**BH** split happens in `parseSrc'`, depending on the filename extension: “.bsv” for **BSV** or “.bs” for **BH**.

File: bsc/src/comp/Depend.hs

```
parseSrc'
| Parser.Classic::pPackage   — BH parse
| Parser.BSV::bsvParseString — BSV parse
```

Historical note:

Originally (ca. 2000), the language was just called **Bluespec**, and there was only one syntax, hence the file extension “.bs”. It had/has Haskell-like syntax, and today we call it “Bluespec Classic” or **BH**.

Later (ca. 2004) we added the **BSV** syntax (SystemVerilog-like) as an alternative.

The only difference between **BSV** and **BH** is the syntax and these two front-end parsers. Both of them produce a **CSyntax** output, after which there is no difference in the rest of the compiler; in particular **BSV** and **BH** share the same semantics. Using **BSV** or **BH** is just a personal preference. A single design can mix packages written in **BSV** with packages written in **BH**

6.1 Parsing BH CParser::pPackage

The **BH** parser top-level:

File: bsc/src/comp/Parser/Classic/CParser.hs

pPackage — Parse into CSyntax

6.2 Parsing BSV CVParseString

The **BSV** parser top-level:

File: bsc/src/comp/Parser/BSV/CVParseString.hs

bsvParseString — Parse, then desugar, into CSyntax

After parsing there are “desugaring” steps which:

- convert from imperative syntax into pure functional form. For example, **BSV** syntax allows a variable to be “updated” in an Action-block, and these are converted into pure functional form.
- for-loops are converted into recursive functions.
- StmtFSM constructs are converted into algebraic types [TODO: confirm?]

This results in some redundant info for the parser, like **Action** and **action** keywords [TODO: clarify].

7 Type checking TypeCheck::cTypeCheck

File: bsc/src/comp/ParseOp.hs

```
cTypeCheck :: ErrorHandle -> Flags -> SymTab -> CPackage -> IO (CPackage,
    Bool, S.Set Id)
```

cTypecheck converts between different variants of **CSyntax**. There are different invariants at different stages.

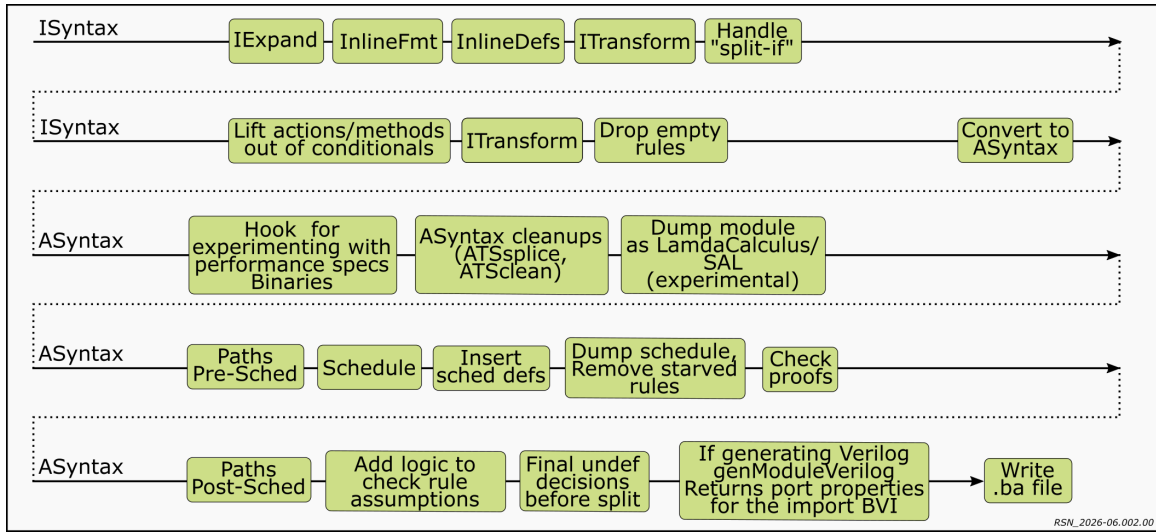


Figure 2: Major stages in flow of function `genModule`

8 Code generation `Main_bsc::genModule`

`genModule` synthesizes a module (generates Verilog or Bluesim code). It, in turn, consists of a number of stages, shown in Figure 2. Internally, it converts `ISyntax` into `ASyntax`.

File: `bsc/src/comp/bsc.hs`

```

genModule
— Input is in ISyntax
  iExpand      — "run" the module monad
  iNormTypes   — normalize types (handle lingering SizeOf)
  iInlineFmt   —
  iInline      — Inline defs
  iTransform   —
  iSplitIf     — Split rules containing if statements
  iLift        — Lift actions/methods out of conds,
               — and merge where possible
  iTransform   — Transform to simpler form (take 2)
  iParams      — Inline expressions for submodule instantiation
               — parameters, or separate them into localparam defs
               — instead of wire defs
  iDropRules   — Drop empty rules
  aConv        — Generate ATS ("Abstract Transition Syntax",
               — cf. James Hoe thesis)

— We are now working with ASyntax (type APackage file ASyntax.hs)

  flattenInstTree
  aRankMethCalls — append ranks to method names and calls for
                  — "performance guarantees"
  aTaskSplice    — splice in identifiers set by ActionValue tasks
                  — into ATaskAction calls
  aCleanup       — clean up ATS (merge any ME calls to same action
                  — in same rule)

  convAPackageToLambdaCalc; dump

```

```

                                — experimental dump to lambda calc (for formal verific)
convAPackageToSAL; dump
                                — experimental dump to SAL (for formal verific)

aPathsPreSched — Build path graph for everything except rules
aSchedule      — Schedule
aAddScheduleDefs
                — Add CAN_FIRE and WILL_FIRE defs based on the schedule
aDumpSchedule
aCheckProofs   — prove any proof obligations accumulated at this point
                — the obligations are removed from the module
aPathsPostSched
                — Add scheduler edges to the path graph
                — (assume pathGraphInfo is consistent with
                — amod_checked, because at most only a few rules
                — have been removed since amod_clean)
aNoInline      — lift all no-inline func calls
                — and assign instance names to them
aAddCFConditionWires,
                — add scheduling assumptions (ME, CF, etc.) to APackage
    aAddSchedAssumps
                — (in two steps)
aRemoveAssumps — move assumption actions into rule bodies
aDropUndet     — drop ASAny with "chosen" values
                — just before the split because other paths add new
                — expressions
                — and because we might choose more undets in
                — pathsPostSched

— The amod doesn't change beyond this point

apkg_external_wires
    — save wireinfo
getAPackageFieldInfos

genModuleVerilog
writeABin      — if generating Verilog
deffun        — if genABin

deffun        — "wrappergen" handling value methods with
                — constant True output (TODO: always ready?)

```

9 Notes on building the *bsc* compiler (compiling and linking)

NOTE: Most *users* of *bsc* do not need to build *bsc* from scratch. Pre-built binaries are available for many Linux platforms and for MacOS; please see the “Download ... Releases” link in the repo’s README.

bsc is built using the **ghc** Haskell compiler. The file `INSTALL.md` in the repo <https://github.com/B-Lang-org/bsc> is the definitive document on how to do this. This section has some additional notes about the process.

To be written ...

A Brief Haskell cheat sheet

We describe here the basic Haskell notations used in the *bsc* source code. This is not intended to be a Haskell tutorial; many such tutorials exist on the web. However, descriptions here may be enough to enable reading the source code.

This section needs to be written ... but the headings below indicate the relevant Haskell features one should know in order to read the source code.

.hs and .lhs (“literate Haskell”) files

Indentation and “off-side rule”

Top-level function type declarations and definitions

Function application, partial application

IO monad

do notation, <- binding, return

let bindings

\$ composition (right-associative)

List notation, list comprehensions

Lambda expressions

if-then-else and case expressions

Haskell modules, imports and exports

NOTE:

A possible point of confusion: What Haskell calls a **module** (a large-program organizing construct) is called a **package** in **BSV/BH** because in **BSV/BH** the word “module” is reserved for hardware items.

A Haskell **module** or **BSV/BH package** allows a large program to be split into smaller pieces (usually each in its own file), with controlled visibility of identifiers across the boundary (using imports and exports, and qualified-names). It is purely a source-code organizing mechanism, and has no hardware significance.

Algebraic type definitions, deriving, show

Typeclasses and instances

The *bsc* code itself does not seem to define any new Haskell typeclasses, just instances of existing typeclasses.